Operations Research 1: Deterministic Models
I/O between Python and Excel

Dr. Lei Zhao

Fall, 2025

Operations & Services Research (TOpS) Laboratory
Department of Industrial Engineering
Tsinghua University, China

Preliminaries

- To follow this tutorial, you should have
  - Set up the Python environment according to the *Python Setup* tutorial
  - Known how to use basic Python data structures
    - List
    - Dictionary
    - …
  - Known basic Python syntax
    - “as”, “range”, “import”, “in” and “is” keywords
    - “if…elif…else…” statement
    - “for” and “while” loops
    - …
- Note that we have provided some Python self-study references at the end of the *Python Setup* tutorial.

Two Useful Python Packages

- We recommend two Python packages to read/write Excel files.
  - Openpyxl
    - A package to read/write Office Open XML format
    - For a detailed description, please refer to its webpage: https://openpyxl.readthedocs.io/en/stable
  - Pandas
    - A fast, powerful, flexible, and easy-to-use open source data analysis and manipulation tool
    - For a detailed description, please refer to its webpage: https://pandas.pydata.org

Two Useful Python Packages: Openpyxl

- Installation and verification
  - Open Anaconda Prompt (Windows OS) or Terminal (Mac OS and Linux OS).
  - Installation: enter the command (run as administrator)

    ```
    pip install openpyxl
    ```

  - Verification: enter the commands

    ```
    python
    from openpyxl import load_workbook
    ```

  - If there is no error, you have successfully installed Openpyxl in your Python environment.

Two Useful Python Packages: Pandas

- Installation and verification
  - Open Anaconda Prompt (Windows OS) or Terminal (Mac OS and Linux OS).
  - Installation: enter the command (run as administrator)

    ```
    pip install pandas
    ```

  - Verification: enter the commands

    ```
    python
    import pandas as pd
    ```

  - If there is no error, you have successfully installed Pandas in your Python environment.

Openpyxl vs. Pandas

- Openpyxl
  - Pros
    - Read and write data cell by cell ➔ very flexible
    - Easily edit the Excel styles (font, color, alignment, etc.)
      - Examples: https://openpyxl.readthedocs.io/en/stable/styles.html
  - Cons
    - Could be cumbersome and slow when you have a lot of data to process
    - Its official documentation is not very easy to follow.

Openpyxl vs. Pandas

- Pandas
  - Pros
    - If the input data are organized in matrices, reading data from an Excel file with Pandas is fairly easy.
    - Good official documentation
  - Cons
    - Cannot write data to an existing Excel sheet
    - Can only write a DataFrame object (like tables) to a new Excel sheet
    - Hard to edit Excel styles (font, color, alignment, etc.)

Openpyxl vs. Pandas

	Flexibility	I/O speed	Style editing	Good docs
Openpyxl	✅		✅	
Pandas		✅		✅

- In this course, we provide exemplary programs for both Openpyxl and Pandas.

Wyndor Example

Wyndor Example

- Wyndor Glass Co.
  - Two products
    - Product 1: an 8-foot glass door
    - Product 2: a 4x6 foot wood-frame window
  - Three plants
  - Limited production time per week (for each plant)
  - “Infinite” demands

	Doors	Windows			
Profit per batch	$3,000	$5,000			
					Hours
	Hours used per batch produced				available
Plant 1	1	0			4
Plant 2	0	2			12
Plant 3	3	2			18
	Doors	Windows			Total profit
Batches produced					

Wyndor Example

- x_1 = batches of product 1 per week
- x_2 = batches of product 2 per week
- Z = total profit per week

	Doors	Windows		
Profit per batch	$3,000	$5,000		
				Hours
	Hours used per batch produced			available
Plant 1	1	0		4
Plant 2	0	2		12
Plant 3	3	2		18

$$
\begin{array}{lrrl}
\max & Z = & 3x_1 \quad +5x_2 & \\
\text{s.t.} & & x_1 & \leq 4, \\
& & 2x_2 & \leq 12, \\
& & 3x_1 \quad +2x_2 & \leq 18, \\
& & x_1, \quad x_2 & \geq 0.
\end{array}
$$

An Exemplary Program

- Note that in this tutorial
  - We omit the formulation and optimization of the LP model using Gurobi in Python, which is elaborated in the tutorial *LP Modeling*.
  - We assume an arbitrary solution and its objective function value.

```
# An arbitrary solution and its objective function value
soln = {"Doors": 0, "Windows": 0}
objVal = 0
```

An Exemplary Program

- main.py
  - The main program to read data from Excel file, compute and then write results to Excel file.
- io_openpyxl.py
  - Use openpyxl for reading and writing.
  - Functions
    - readDataOpenpyxl(): read the data from Excel file
    - writeDataOpenpyxl(): write the data to Excel file
- io_pandas.py
  - Use pandas for reading and writing.
  - Functions
    - readDataPandas(): read the data from Excel file
    - writeDataPandas(): write the data to Excel file
- constant.py
  - Provide names and values of related parameters.

Basic Functions in Openpyxl and Pandas

Openpyxl: Read Data from an Excel file

- Function openpyxl.load_workbook()
  - Important input parameter
    - filename: the path to your Excel file
  - Return value
    - A Workbook object that connects to the input Excel file
      - To get a Worksheet object that connects to a sheet of the input Excel file, you can use `Workbook['SHEET_NAME']`.
      - For descriptions of the Workbook and Worksheet data structures, see https://openpyxl.readthedocs.io/en/stable/api/openpyxl.workbook.workbook.html https://openpyxl.readthedocs.io/en/stable/api/openpyxl.worksheet.worksheet.html
  - Examples: https://openpyxl.readthedocs.io/en/stable/usage.html?highlight=load_workbook#read-an-existing-workbook

Openpyxl: Read Data from an Excel File

- Function Worksheet.cell()
  - Important input parameters
    - row: row index of the cell
    - column: column index of the cell
  - Return value
    - Value of the cell located at (row, column)

Openpyxl: Write Data to an Excel File

- Function Worksheet.cell()
  - Important input parameters
    - row: row index of the cell
    - column: column index of the cell
    - value: value assigned to the cell
  - We use this function to write data to cell located at (row, column).

Pandas: Read Data from an Excel File

- Function pandas.read_excel()
  - Some important input parameters
    - io: the path to your Excel file
    - sheet_name: the sheet you read
    - usecols: the columns you parse
    - skiprows: # lines to skip at the start of the sheet
    - nrows: # rows to parse
  - Return value
    - A DataFrame object that contains the data you specify by input parameters
      - For description of the DataFrame data structure, see https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html
  - Other details & examples: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_excel.html

Pandas: Write Data to an Excel File

- Function pandas.ExcelWriter()
  - Some important input parameters
    - path: the path to your Excel file
    - engine: the engine used for writing (e.g., Openpyxl)
    - mode: file mode to use (write or append)
  - Return value
    - An ExcelWriter object
  - Other details & examples: https://pandas.pydata.org/docs/reference/api/pandas.ExcelWriter.html

Pandas: Write Data to an Excel File

- Function DataFrame.to_excel()
  - Some important input parameters
    - excel_writer: an ExcelWriter object or the path to your Excel file
    - sheet_name: the sheet you write
    - startrow: upper left cell row to dump the data frame
    - startcol: upper left cell column to dump the data frame
  - We use this function to write a given DataFrame object to an Excel sheet.
  - This function cannot write data to an existing Excel sheet.
  - Other details & examples: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.to_excel.html